

avito

Классификация типа комнаты по фото

Автоматическая разметка фотографий
в объявлениях недвижимости

Артем Власов · Владимир Сызранов · Константин Гашев

Май 2026 · МИФИ × Avito

Macro F1

0.7565

лучший результат
на тесте

19 классов · 48 003 предсказания

Постановка задачи

Зачем это нужно и что мы классифицируем

Задача

- Многоклассовая классификация изображений
- 1 фото → 1 класс из 19 типов помещений
- Метрика качества: Macro F1 (равный вес каждому классу)
- Датасет: соревнование Avito × МИФИ

Применение в продукте

- Автоматическая разметка фото в объявлениях
- Контроль качества карточек недвижимости
- Улучшение поиска и фильтрации по объектам

19 классов помещений

0 — кухня / столовая	10 — коридор / прихожая
1 — кухня-гостиная	11 — гардеробная
2 — гостиная	12 — балкон / лоджия
3 — спальня	13 — вид из окна
4 — детская	14 — дом снаружи
5 — кабинет	15 — подъезд
6 — ванная	16 — другое
7 — туалет	17 — предметы интерьера
8 — санузел совм.	18 — без мебели

Этапы работы над проектом

3 фазы от baseline до продуктового сервиса

1

Фаза 1

Базовое решение

- Анализ данных и классов
- Обучение EfficientNet-B0
- Macro F1: 0.6093
- Выявление проблемных классов



2

Фаза 2

Улучшение данных и модели

- Ручная доразметка 687 фото
- Балансировка классов
- ConvNeXt Base + TTA
- 3-fold ансамбль, F1: 0.7559



3

Фаза 3

Продуктовая упаковка

- Web-интерфейс Gradio
- Docker-контейнеризация
- README, .gitignore, .env
- Stacking, submission 48 003

Данные: расширение и балансировка

Ключевое решение Фазы 2 для улучшения Macro F1

Проблема

- Дисбаланс классов в исходном датасете
- Редкие классы: кабинет, другое, универсальная, предметы интерьера
- Визуальная близость пар: ванная/санузел/туалет, гостиная/спальня

Решение

- Ручная доразметка изображений командой
- Доменная фильтрация внешних источников
- Скр по классам для предотвращения перекоса

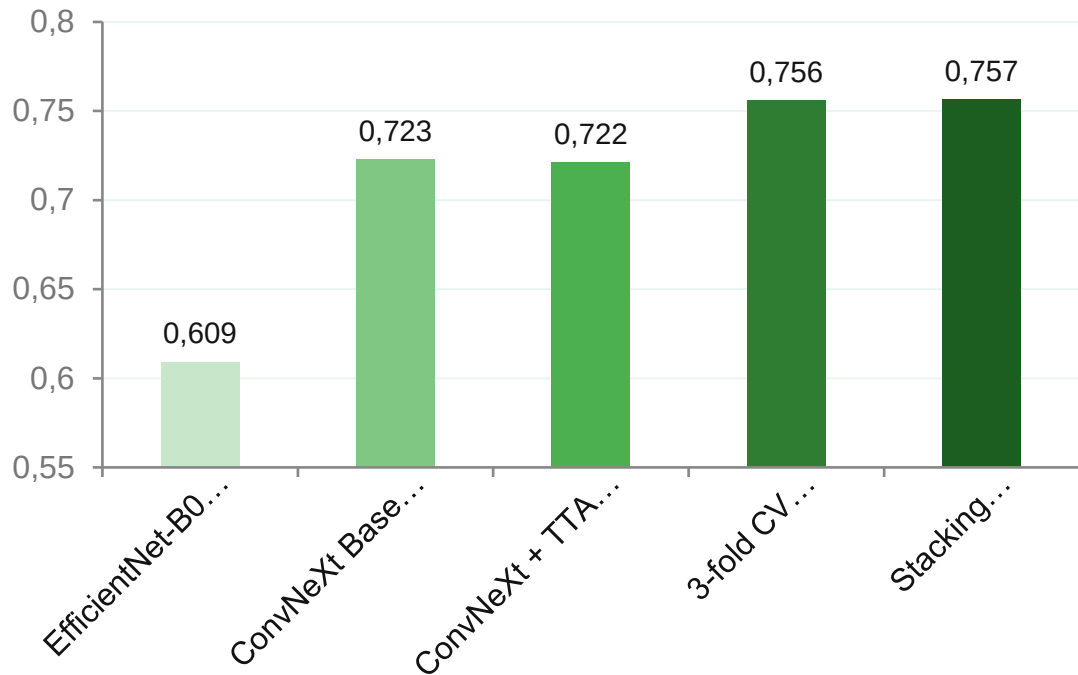
Вклад команды (Фаза 2)

Участник	Класс	Фото
Артем	кабинет	118
Артем	другое	89
Владимир	кухня-гостиная	170
Владимир	предметы интерьера	70
Константин	универсальная комната	180
Константин	предметы интерьера	60

Итого добавлено: 687 изображений

Модель и эксперименты

Хронология развития качества, Macro F1



Порог оценки: 0.60

Ключевые решения

LR sweep

$lr=1e-4$ — оптимальный

Архитектура

ConvNeXt Base >> EfficientNet

TTA

flip-TTA: +0.003

3-fold CV

mean F1: 0.7559, std ± 0.013

Stacking

OOF meta-model, F1: 0.7565

Итоговые результаты

avito

0.7565

Macro F1
финальный

Stacking ансамбль

19

Классов
помещений

Полное покрытие

48 003

Строк
submission

Финальный тест

× 1.24

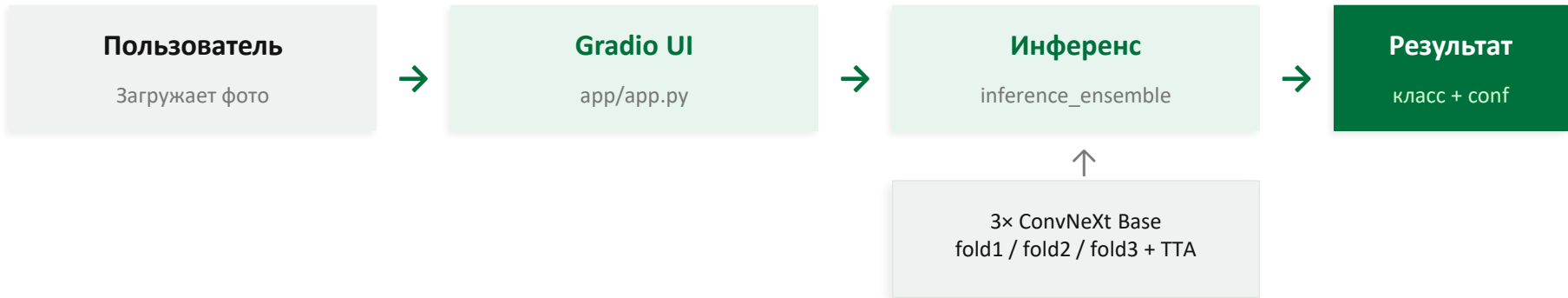
Прирост
к baseline

F1: 0.60 → 0.76

Этап	Модель / подход	Macro F1
Фаза 1 — baseline	EfficientNet-B0	0.6093
Фаза 2 — одиночная	ConvNeXt Base (class-focus)	0.7228
Фаза 2 — ансамбль	3-fold ConvNeXt, mean	0.7559
Фаза 2 — финал	Stacking no OOF	0.7565 ✓

Архитектура продуктового сервиса

Компоненты финального решения



Web-интерфейс

Gradio, порт 7860
Локальный инференс



Docker

Dockerfile
docker-compose.yml



Модели

3x best_model.pth
Stacking artifacts



Git-публикация

.gitignore, .gitattributes
Git LFS для весов

Технологический стек

avito

Open-source, локальный инференс, без внешних API

ML / Обучение

- PyTorch + CUDA (RTX 3060)
- timm (ConvNeXt Base)
- scikit-learn (stacking)
- numpy, pandas

Данные

- Pillow (обработка фото)
- Ручная разметка (687 img)
- Stratified 3-fold CV
- Test-Time Augmentation

Сервис и UI

- Gradio (web UI)
- Docker + Docker Compose
- Python venv
- localhost:7860

Публикация

- Git + Git LFS (веса)
- .gitignore, .gitattributes
- .env.example
- GIT_PUBLISHING_GUIDE.md

Интерфейс пользователя и запуск сервиса

avito

Gradio UI · Docker · localhost:7860

1



2



3



4

Открыть
<http://localhost:7860>

Загрузить
фото комнаты

Нажать
«Определить»

Получить
тип + уверенность



Модель работает локально. Загруженные изображения не отправляются во внешние сервисы.

Запуск через Docker

```
docker compose up --build
```

<http://localhost:7860>

Локальный запуск (без Docker)

```
pip install -r app/requirements.txt
```

```
python app/app.py
```

Роли и вклад команды

Тимлид: Власов Артём | Сызранов Владимир · Гашев Константин

А

Артём Власов

Тимлид · ML / Prod Engineer

- Ф.1 Создание и обучение моделей (baseline)
EfficientNet-B0, настройка GPU/CUDA 12.8
Сбор данных: 118 кабинет + 89 другое
- Ф.2 Создание инференса 3-fold ансамбля
Stacking pipeline (LogReg на OOF)
- Ф.3 Финальная сборка prod-сервиса
Gradio UI/UX, Docker, deploy
Финальный отчёт, презентация

В

Владимир Сызранов

Data / ML инженер

- Ф.1 Участие в EDA, подготовка датасета
Сбор данных: 170 кухня-гостиная
Сбор данных: 70 предметы интерьера / быт. техника
- Ф.2 Участие в обучении ConvNeXt
Валидация предсказаний и метрик
- Ф.3 Участие в финальной проверке
Подготовка к защите

К

Константин Гашев

ML Engineer · Модели

- Ф.1 Сбор данных: 180 универсальная комната
Сбор данных: 60 предметы интерьера / быт. техника
- Ф.2 Создание ConvNeXt Base моделей
3-fold CV, LR sweep (оптим. lr=1e-4)
Отчёт по экспериментам (handover)
- Ф.3 Верификация артефактов
Передача весов и конфигов

План работ — факт выполнения

Сводка: что сделано, кем, какой результат получен

Этап	Выполненные работы	Кто	Результат / артефакт
0. Инфраструктура и EDA	Kick-off, Git-репо, структура проекта EDA: 4 562 изображений, дисбаланс классов Выявлены трудные классы (кабинет×74, гардеробная×59)	Команда	<i>train_df.csv, EDA-омчём, /src /configs /models</i>
1. Сбор данных (Фазы 1–2)	Ручная разметка 687 фото по трудным классам: AB: 118 кабинет + 89 другое BC: 170 кухня-гостиная + 70 предметы КГ: 180 универсальная + 60 предметы Фильтрация + balancing	AB+BC+КГ	<i>train_df_full_class_focus.csv (4 598 строк)</i>
2. Baseline (Фаза 1)	EfficientNet-B0, transfer learning, исправление путей LR sweep exp2/exp3/exp4	AB	<i>Macro F1 = 0.6093 ✓ best_model.pth</i>
3. ConvNeXt Base (Фаза 2)	ConvNeXt Base (timm), GPU RTX 3060, CUDA 12.8 LR sweep: lr=1e-4 → F1=0.7176 class-focus retrain → F1=0.7228	КГ + AB	<i>configs/*.yaml fold1–3 checkpoints (~350 МБ)</i>
4. 3-fold CV + Stacking (Фаза 2)	Stratified 3-fold: 0.7378 / 0.7595 / 0.7704 mean=0.7559 Flip-TTA +0.003 Stacking LogReg на OOF → F1=0.7565	КГ + AB	<i>best_model.pth ×3 oof_meta.csv submission.csv 48 003</i>
5. Prod-сервис (Фаза 3)	Gradio UI (app.py), локальный инференс, порт 7860 Dockerfile + docker-compose .env .gitignore Git LFS	AB	<i>localhost:7860 Docker-образ готов</i>
6. Документация и защита	README.md FINAL_PROJECT_REPORT.md EXPERIMENT_REPORT.md Презентация .pptx / .pdf Итоговый документ в ЛК	AB	<i>Все документы в /docs Итоговый документ в ЛК</i>

Интерфейс сервиса — демонстрация

avito

Gradio UI · localhost:7860 · локальный инференс

Распознавание типа комнаты по фото

Загрузите фотографию помещения. Сервис применит лучшую модель проекта: ансамбль трех ConvNeXt Base-моделей с локальным инференсом.



☒ Использовать horizontal flip TTA

Определить тип комнаты

Top классов

3 - гостиная

3 - гостиная	61%
10 - коридор / прихожая	17%
2 - универсальная комната	6%
18 - комната без мебели	5%
1 - кухня-гостиная	3%

Детали инференса

Предсказание: 3 - гостиная
Уверенность: 0.614
Модель: 3-fold ConvNeXt Base ensemble
TTA: none, hflip
Устройство: cpu
Время обработки: 959 mc
Вес: E:\Python\MIF\prectice2curse\avito_solution_bundle\weights\fold1\best_model.pth,
E:\Python\MIF\prectice2curse\avito_solution_bundle\weights\fold2\best_model.pth,
E:\Python\MIF\prectice2curse\avito_solution_bundle\weights\fold3\best_model.pth

Модель работает локально. Загруженные изображения не отправляются во внешние сервисы. В UI отображаются 19 классов задания; дополнительный обучающий класс '19' объединяется с классом '18' - комната без мебели.

Gradio web UI · Загрузка фото · Топ-5 классов с уверенностью

Соответствие критериям оценки

Все ключевые требования выполнены



Подготовка данных

Очистка, балансировка, ручное расширение датасета



Обучение модели

Baseline EfficientNet-B0 + ConvNeXt Base (3-fold)



Расчёт метрик

Macro F1 по всем экспериментам и фолдам



Качество классификации

Macro F1 = 0.7565 (порог ≥ 0.60 ✓)



Финальный submission

48 003 строки для тестового набора



Прототип интерфейса

Web-интерфейс Gradio с загрузкой фото



Разделение train / infer

Код обучения, конфиги и инференс разделены



Локальный запуск

Инференс без внешних API, localhost:7860



Контейнеризация

Dockerfile + docker-compose.yml



Документация

README, отчёт, GIT_PUBLISHING_GUIDE.md

Ограничения и направления развития

Текущие границы и что можно улучшить

Текущие ограничения

- Инференс на CPU медленнее, чем на GPU
- Качество зависит от схожести фото с обучающим доменом
- Веса моделей требуют Git LFS для публикации
- Нет REST API для промышленной интеграции
- Пакетная обработка не реализована

Направления развития

- FastAPI-слой для интеграции с внешними системами
- Batch-инференс endpoint
- MLflow / W&B для логирования экспериментов
- Grad-CAM: объяснение предсказаний
- Публикация модели в Hugging Face

Выводы

avito

F1

Цель превышена

Macro F1 = 0.7565 — выше порога 0.60 на 26%. Стабильный 3-fold CV, std ± 0.013 .

DB

Данные решают

Ручная доразметка 687 фото по трудным классам дала ключевой прирост качества.

PR

Готово к демонстрации

Сервис упакован в Docker, запускается одной командой, работает без внешних API.

TM

Командная работа

Чёткое распределение по ролям: разметка, обучение, упаковка. Все этапы закрыты.